

Debian Trixie (12) on 2017 iMac 5K and 2019 MacBook Pro – Complete Setup Guide

Overview and Preparation

Running Debian 12 “Trixie” on Apple hardware requires careful preparation to ensure all components work. The 2017 iMac 27" 5K and the 2019 MacBook Pro (with Touch Bar and T2 chip) have unique hardware (Broadcom/Apple chipsets, AMD GPUs, T2 security chip, etc.) that need extra drivers and firmware. We will use the MATE desktop environment, and enable **non-free** repositories to install proprietary drivers/firmware as needed. Before starting:

- **Backup & Decide on Dual-Boot vs. Wipe:** If keeping macOS, use Disk Utility to shrink the macOS partition and leave free space for Debian. If replacing macOS, you can repurpose the entire drive for Debian (the Debian installer will handle partitioning).
- **T2 Security Settings (MacBook Pro 2019 only):** On T2-equipped Macs, you **must** adjust settings to allow Linux to boot. Reboot into macOS Recovery (hold `Cmd+R` at startup), open **Startup Security Utility**, and **disable Secure Boot** or set it to “No Security”. Also **allow booting from external/unsigned OS**. This permits the Debian installer USB to load.
- **Create Debian 12 Installer USB:** Download the **Debian 12 “Bookworm” (Trixie)** AMD64 ISO. It’s recommended to use the **“non-free firmware”** included ISO so that common firmware (e.g. for GPUs or Wi-Fi) is available during install. Write the ISO to a USB drive.
- **External Peripherals:** Have a **USB keyboard and mouse** ready, especially for the MacBook Pro. The MacBook’s internal keyboard/trackpad won’t work in the installer without special drivers (since they interface via the T2 chip). An external keyboard/mouse will let you navigate the installer. For internet, if possible connect an Ethernet cable (the iMac has an Ethernet port; the MacBook Pro will need a USB-C Ethernet adapter) or prepare a USB Wi-Fi adapter or phone tethering, since the internal Wi-Fi won’t work until after post-install configuration.

With preparation done, we’ll cover installation and post-install setup for each Mac model separately.

Installing Debian on the 2017 iMac 5K

Step 1: Booting the Installer on iMac

1. **Boot from USB:** Insert the Debian USB and reboot the iMac. Hold the **Option (⌥)** key at startup to invoke the boot picker. Select the USB (usually labeled “EFI Boot”) to start the Debian installer.
2. **Choose UEFI Mode:** The iMac supports UEFI, which Debian will use by default. Proceed with the graphical installer or text installer as preferred.
3. **Keyboard/Mouse in Installer:** You can use the iMac’s USB or Bluetooth keyboard/mouse. If using Apple wireless peripherals, connect them via Lightning cable to ensure they function as USB HID devices during install (Bluetooth may not be active in the installer). Alternatively, use any USB keyboard/mouse.

Step 2: Debian Installation on iMac

1. **Partitioning:** In the installer, when you reach disk partitioning, decide your layout:
2. For **dual-boot**: Select the free space you prepared and create at least a root (`/`) partition (ext4 or Btrfs). Also create a **swap** partition if desired (or use a swap file later). Do **not** delete the existing EFI System Partition (ESP) used by macOS. Mount the existing ESP at `/boot/efi` during setup so GRUB can install there.
3. For **whole-disk**: You can let Debian auto-partition the entire disk. This will create an EFI partition (if not present) and the necessary Linux partitions. (Be aware this will erase macOS and all data on the drive.)
4. **Install GRUB to EFI:** When prompted to install a bootloader, choose **GRUB-EFI**. Point it to the existing EFI partition (on a Mac, this is typically a ~200MB FAT32 partition, often `/dev/sda1` or `/dev/nvme0n1p1`). The installer should detect it and install Debian's bootloader there. *(If the installer fails at the GRUB step – a known issue if the ISO's kernel is old – you may need to chroot into the install and manually install grub after, but with Debian 12 ISO this is unlikely)* ¹ ² .
5. **Finishing Up:** Complete the installation steps (set up users, etc.). Once done, reboot. Hold **Option** on reboot and select “EFI Boot” (or it may show as “Debian”) to boot into your new Debian system ³ . On non-T2 Macs, you can also set Debian as the default boot OS via `sudo efibootmgr` if desired, or use rEFInd for a nicer boot menu (optional).

Step 3: Post-Install Configuration on iMac – Drivers & Firmware

After first boot, the iMac will likely be missing Wi-Fi, Bluetooth, and possibly audio. We'll enable those now. Log in (you may need a wired keyboard if Bluetooth isn't up yet) and ensure the “**contrib**” and “**non-free**” components are enabled in `/etc/apt/sources.list` for Debian. Then run `sudo apt update`.

- **Wi-Fi (Broadcom):** The 2017 iMac 5K uses a Broadcom Wi-Fi chipset (e.g. BCM4364) that isn't supported by the default kernel driver without proprietary components ⁴ . Debian's **broadcom-sta (wl) driver** supports many Apple Broadcom chips. Install it by running:

```
sudo apt install broadcom-sta-dkms firmware-brcm80211
```

This installs the Broadcom STA driver (as a DKMS module) and Broadcom firmware. Next, disable the conflicting open-source drivers (which don't work for this chip) and load the STA module:

```
sudo modprobe -r b43 b43legacy bcma brcmsmac ssb
sudo modprobe wl
```

If Secure Boot is disabled, the module should load (if Secure Boot is still on, the unsigned module will be blocked – you'd need to sign it or keep Secure Boot off). After loading, you should see the Wi-Fi interface (e.g. via `iwconfig` or NetworkManager). Use NetworkManager (MATE's network applet) to connect to Wi-Fi.

Alternative Wi-Fi method: Instead of the Broadcom STA driver, you can use the open-source **brcmfmac** driver with Apple's firmware. This requires extracting firmware from Apple's Boot Camp drivers or macOS. The community T2Linux project provides a script/package called **apple-firmware-script** to automate this ⁵ . On a non-T2 iMac, this is optional – the STA driver method above is

simpler. But if you prefer: install the script (`sudo apt install apple-firmware-script` from the T2 repo, explained later) and run `sudo get-wifi-firmware`. This will download Apple's Wi-Fi firmware and install it for use with `brcmfmac`. Then load `brcmfmac` with `sudo modprobe brcmfmac`. Either approach will enable the Broadcom Wi-Fi. (If Wi-Fi connects but cannot pass traffic or repeatedly asks for password, it may be a wpa_supplicant issue; using an alternative like `iwd` or adding the kernel param `brcmfmac.feature_disable=0x82000` can work around a known Broadcom issue ⁶ ⁷.)

- **Bluetooth:** The iMac's Bluetooth is part of the same Broadcom card. Once the Broadcom firmware is in place (via `firmware-brcm80211` or the above script), the Bluetooth adapter should be recognized by the kernel **btusb** driver. Install `firmware-brcm80211` (if not already from above) which includes some Broadcom BT firmware. Then load Bluetooth: `sudo modprobe btusb`. Use a tool like **blueman** (MATE's Bluetooth manager) to pair devices. If Bluetooth still isn't working, use the `apple-firmware-script` which also pulls in Bluetooth firmware, or manually extract the firmware from Boot Camp drivers. Note that on some Broadcom chip versions (e.g. BCM4377 in newer Macs) Bluetooth may have stability issues on 2.4GHz Wi-Fi bands ⁸, but on the 2017 iMac's chip this is usually fine.
- **GPU (Retina 5K Display & Acceleration):** The 2017 iMac 5K comes with an AMD Radeon Pro GPU (e.g. Radeon Pro 570/575/580). Debian's open-source **AMDGPU driver** supports this. Install the **firmware-amd-graphics** package to ensure the latest microcode is present:

```
sudo apt install firmware-amd-graphics
```

This provides the necessary GPU firmware for full acceleration. The Xorg driver (`xserver-xorg-video-amdgpu`) should already be installed by default. After installing the firmware, run `sudo update-initramfs -u` and reboot. You should have proper 5K resolution and GPU acceleration (verify with `glxinfo | grep "OpenGL renderer"` – it should list your AMD GPU). MATE might appear tiny at 5K resolution; you can enable HiDPI scaling in MATE Appearance settings or via `dconf` (e.g. set `org.mate.desktop.interface scaling-factor` to 2) to make UI elements a comfortable size.

- **Audio (Cirrus Logic Codec):** The iMac's internal speakers and microphone are connected via an Apple custom Cirrus Logic audio codec (likely CS8409). Out of the box, you might see “Dummy Output” for sound. Newer Linux kernels have partial support for this codec, but Debian 12's default kernel (6.1.x) may still require a quirk. First, ensure the **alsa firmware** package is installed: `sudo apt install firmware-intel-sound` (covers DSP firmware, though Apple's codec is custom). Reboot and check if audio devices appear (e.g. `aplay -l`). If not, you'll need a patch driver. The community has created a custom kernel driver for the Cirrus 8409 on Macs ⁹. One solution is to compile the driver from the [davidjo/snd_hda_macbookpro GitHub project](#) which provides support for CS8409+AMPs on 2017+ iMacs and MacBook Pros. Installing this as a DKMS module can enable the speakers and mic properly. (As of kernel 6.3+, much of this might be upstreamed, so check if upgrading the kernel helps before custom compiling.) After installing the driver or upgrading the kernel, set the correct ALSA model if needed (e.g., in `/etc/modprobe.d/alsa-base.conf` you might set `options snd_hda_intel model=imac27` for the iMac27, though newer drivers autodetect). Use `alsamixer` to adjust volumes; unmute the speakers and mic. Once recognized, audio output should work. (If the iMac has a headphone jack, test that as well. Some Apple combos need additional quirks for headphone sensing – refer to the Debian or Alsa documentation if needed.) ¹⁰

- **Camera (FaceTime HD Webcam):** The built-in iSight/FaceTime HD camera in newer iMacs is not a standard USB webcam – it's actually a PCIe Broadcom 1570 camera that needs a special driver. By default, Debian won't see a `/dev/video0` for it ¹¹. To enable the webcam, install the open-source **facetimehd** driver and firmware. This involves two components: firmware (extracted from Apple's driver) and the kernel module. The steps are:

```
# Install build tools and headers
sudo apt install git build-essential dkms linux-headers-$(uname -r)
# Download and install the firmware
git clone https://github.com/patjak/facetimehd-firmware.git
cd facetimehd-firmware && make && sudo make install && cd ..
# Build and install the driver
git clone https://github.com/patjak/bcwc_pcie.git
cd bcwc_pcie && make && sudo make install
sudo depmod -a
```

You can also set this up as a DKMS package so it rebuilds on kernel updates (some distributions provide a `facetimehd-dkms` package ¹², but on Debian you'll likely compile manually as above). Once installed, load the module: `sudo modprobe facetimehd`. The camera should now be available (test with an application like Cheese or `ffmpeg -f v4l2 -i /dev/video0 -frames:v 1 test.jpg`). Keep in mind this driver is reverse-engineered and might not support all camera features, but basic 720p video should work.

- **Apple Keyboard & Hotkeys:** If you use an Apple USB or Bluetooth keyboard, the **hid_apple** kernel module drives it. Debian should load it automatically. You can customize behavior: for example, by default media keys (play, volume, etc.) should work. The function keys (F1...F12) on Apple keyboards might be mapped to special functions without pressing "Fn". To invert that (so F1-F12 are primary), add a modprobe option:
`echo options hid_apple fnmode=2 | sudo tee /etc/modprobe.d/hid_apple.conf`. This sets `fnmode 2` (make Fn keys standard and use Fn key to get media keys) ¹³. You can also swap the Command and Control key positions in software if needed (MATE Keyboard Settings or using `xmodmap`). Keyboard backlight is not applicable on the iMac's external keyboard (only laptops have backlit keys).
- **Magic Trackpad / Mouse:** If you use a Magic Trackpad 2 or Magic Mouse with the iMac, they should work via Bluetooth once paired. The Magic Trackpad is treated as a multi-touch device – Linux's **hid_magicmouse** module supports basic functionality (moving cursor, clicking, two-finger scrolling). For more advanced gestures (pinch, swipe with 3+ fingers), consider using a userspace tool like **libinput-gestures** or **fusuma** to define custom gestures. The Magic Mouse will support scrolling (by finger sliding) via the same driver. Install `xserver-xorg-input-libinput` (should already be present) for MATE to handle the input.
- **Power Management:** The iMac is a desktop, so battery management isn't relevant. However, you still have sleep/suspend. Suspending an iMac under Linux can be hit-or-miss. If you choose **Suspend**, the iMac may enter sleep but sometimes immediately wake up (due to USB devices or bluetooth). If you encounter **immediate wake-ups**, you can disable wake triggers: e.g., run `sudo grep . /proc/acpi/wakeup` to see devices – if you see XHC1 (USB controller) enabled, disable it with `echo XHC1 | sudo tee /proc/acpi/wakeup` (and similarly `LID0` for lid sensor on laptops, though iMac has no lid) ¹⁴. This prevents USB events from waking the system. To make this

permanent, you can add those echo commands to `/etc/rc.local` or a systemd service. Fan control on iMac is handled by the SMC – the **applesmc** kernel module should load automatically to expose fan speed and temperature sensors. The SMC usually auto-regulates fan speed to keep thermals in check, but if you find the fan is running high or not responding, you can use a daemon like **macfanctld**. Install it (`sudo apt install macfanctld`) and it will adjust fan speeds based on CPU/GPU temps. Check that the applesmc module is present (`lsmod | grep applesmc`). For display brightness, on iMac the display brightness is controlled by the GPU driver. You can adjust it using software (e.g., MATE Power Manager brightness slider or `xbacklight` if configured). If the brightness keys on your Apple keyboard (if present) don't work by default, you might need to map them manually using MATE's keyboard shortcuts to change `/sys/class/backlight/amdgpu_b10/brightness`.

Troubleshooting on the iMac

- **Wi-Fi Still Not Detected:** If after installing `broadcom-sta-dkms` (`wl` driver) the Wi-Fi interface doesn't show up, verify Secure Boot is off (or enroll the module via MOK). Also ensure the module is actually loaded (`lsmod | grep wl`). If not, try `sudo modprobe wl` and check `dmesg` for messages. It might also be that this Broadcom chip ID wasn't in the driver's supported list; in that case, the `brcmfmac` + firmware approach may work better.
- **No Audio Output or "Dummy" device:** Double-check that the custom driver (or newer kernel) is in use: `cat /proc/asound/cards` should list an "Intel PCH" card for the Cirrus logic codec. If it's listed but still no sound, you may need to manually unmute and increase volume on the relevant mixer channels using `alsamixer` (e.g., unmute the Speaker and Headphone if present). If the card isn't listed at all, the CS8409 driver didn't load – check `dmesg | grep snd_hda` for clues. Installing the `davidjo` driver via DKMS and rebooting is a reliable fix ⁹.
- **Retina 5K resolution issues:** If the display is stuck at a lower resolution or performance is poor, it indicates the AMDGPU driver or firmware didn't load. Install the `firmware-amd-graphics` package as mentioned, and ensure that the kernel is using `amdgpu` (check `lsmod | grep amdgpu`). For high DPI scaling, adjust MATE settings as discussed.
- **Bluetooth pairing problems:** If Apple Bluetooth devices won't pair, try using `bluetoothctl` (CLI) to pair and trust the device. Some Apple peripherals might require entering a PIN code displayed on screen. Also verify that the **bcm** firmware for BT was loaded (`dmesg | grep -i bluetooth` and look for firmware load). In rare cases with certain chip versions, you may need to obtain a specific BT firmware file (e.g., `BCM4364B3.bin` or similar) and place it in `/lib/firmware/brcm/`. The `apple-firmware-script` can fetch these from macOS if needed.
- **Sleep/Wake issues:** If the iMac refuses to stay suspended, use the `/proc/acpi/wakeup` trick above to disable wake from devices. If it doesn't wake up at all (uncommon on iMac), you might have to do a hard shutdown – in such case, consider not using suspend. You can still use screen blanking and CPU power saving without full suspend.

Following the above steps, the 2017 iMac should have functional Wi-Fi, Bluetooth, full 5K graphics with acceleration, working sound, camera, and input devices under Debian 12. Now, let's set up the 2019 MacBook Pro which has additional complexities due to the T2 chip and Touch Bar.

Installing Debian on the 2019 MacBook Pro (Touch Bar, T2 Chip)

The 2019 MacBook Pro (15"/16") presents a tougher challenge: its keyboard, trackpad, Touch Bar, and internal audio are managed by the Apple **T2 security chip**, which Linux needs special drivers for. We will use the community-provided T2 patches to get everything working. Many steps are similar to the iMac (partitioning, etc.), so we will focus on T2-specific differences.

Step 1: Booting the Installer on MacBook Pro

1. **USB Boot & External Keyboard:** As with the iMac, plug in the Debian USB and hold **Option** (⌥) on boot to select it. Remember, at this point the **internal keyboard and trackpad will NOT work** in the installer (the T2 chip doesn't expose them without OS drivers). So ensure you have a USB keyboard/mouse connected.
2. **Installer Options:** You might also notice that the MacBook's internal SSD is detected normally by the installer (T2 presents it as an NVMe interface – kernel 5.4+ handles it ¹⁵). If the installer cannot see the SSD, it's likely a very old kernel – but Debian 12's installer should be fine. Proceed with the installation similar to above: select language, etc.

Step 2: Installing Debian on MacBook Pro (T2)

1. **Partitioning (Dual-boot vs Full):** For dual-boot, use macOS's Disk Utility to create space beforehand. Apple's drives are APFS formatted; shrink the APFS container and leave free space. In Debian installer, create a new ext4 partition (for `/`) in the free space and a swap partition (or plan for swap file). **Important:** Do **not** reformat or remove the existing EFI System Partition – you will reuse it. For a full wipe, let the installer use the whole disk (it will make a new GPT with an ESP).
2. **Install to ESP:** When installing GRUB, point it to the existing EFI partition (usually the installer auto-detects this as `nvme0n1p1` on T2 Macs). The installation of GRUB might succeed but **be aware:** On a T2 Mac, the Mac's firmware might not add Debian to the blessed boot entries automatically. If after installation you reboot and don't see "EFI Boot" for Debian when holding Option, you might need to boot into macOS Recovery and use **Startup Disk** to select the "EFI Boot" (Debian) or use `rEFInd`. In most cases, holding Option will show an "EFI Boot" which is Debian's GRUB; select it to boot Debian.
3. **First Boot (Limitations):** After install, when you first boot into Debian on the MacBook Pro, **do not panic** that keyboard, touchpad, Wi-Fi, sound, etc., are not working yet – this is expected. Log in using the external keyboard (and mouse). We will now add the needed drivers. Ensure you have an internet connection via a USB-C Ethernet adapter or USB tethering (since Wi-Fi isn't up yet).

Step 3: Enabling T2 Hardware Support (Post-Install)

At this point, the system is up, but many devices aren't functional because the T2 chip needs custom drivers. The community **T2Linux** project provides a pre-compiled kernel with T2 patches and an apt repository of drivers ¹⁶ ¹⁷. We will use these to save time. You have two main options:

Option A: Install the T2 Custom Kernel (Recommended) – This provides built-in support for T2 devices in one step.

Option B: Install individual T2 drivers (DKMS modules) – Use this if you prefer to stick to Debian's stock kernel, but it's more involved.

We'll describe Option A (easier), then Option B if needed.

Option A: Using the T2 Linux Kernel

1. **Add T2 APT Repository:** The T2Linux maintainers host an apt repo for Debian/Ubuntu containing the custom kernel and drivers. Add the repo by running:

```
# Add T2 repo GPG key
curl -s --compressed "https://adityagarg8.github.io/t2-ubuntu-repo/KEY.gpg" |
gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/t2-ubuntu-repo.gpg >/dev/null
# Add repo list for Debian 12 (bookworm)
echo "deb [signed-by=/etc/apt/trusted.gpg.d/t2-ubuntu-repo.gpg] https://
github.com/AdityaGarg8/t2-ubuntu-repo/releases/download/bookworm ./" | sudo
tee /etc/apt/sources.list.d/t2.list
sudo apt update
```

(Replace `bookworm` with `testing` in the above if you are on Debian testing/unstable.) ¹⁸ ¹⁹

2. **Install T2 Kernel and Audio Config:** Now install the custom kernel package and related configs:

```
sudo apt install linux-t2 apple-t2-audio-config
```

This will fetch a Linux kernel with T2 patches (and Apple hardware drivers built-in) and configuration for audio ²⁰ ²¹. The `apple-t2-audio-config` provides UCM files for the T2's audio device to help configure the speakers/mic.

3. **Reboot into T2 Kernel:** Reboot the system (`sudo reboot`). In GRUB, select the new `linux-t2` kernel if it's not default. Once booted, the **internal keyboard and trackpad should now work**, as well as the Touch Bar (in its basic "function keys" mode), and the T2 audio device (speakers and mic) should be visible (though we'll tweak audio shortly). The T2 chip essentially exposes devices via the `apple-bce` driver now. (On T2, the keyboard/trackpad appear as USB input devices provided by the `apple-bce` bridge driver ²².)

4. **Wi-Fi and Bluetooth:** The MacBook Pro's internal Wi-Fi (Broadcom BCM4364) still needs firmware. We can use the same **apple-firmware-script** approach here. The T2 apt repo we added includes `apple-firmware-script` (which provides the `get-wifi-firmware` tool ⁵). Run:

```
sudo apt install apple-firmware-script
sudo get-wifi-firmware
```

This script will offer multiple methods to obtain the Broadcom Wi-Fi/Bluetooth firmware. The simplest is **Method 5**, which downloads a macOS Recovery image from Apple and extracts the Wi-Fi/Bluetooth firmware automatically ²³. Choose method 5 and let it run (ensure you have internet via Ethernet/tethering for the download). Once done, it places firmware binaries in `/lib/firmware/brcm/`. Now load the Wi-Fi driver: `sudo modprobe brcmfmac`. The Broadcom interface should come up (check `dmesg` for `brcmfmac` loading firmware successfully). You can then use NetworkManager to connect to Wi-Fi. (If you prefer the Broadcom STA driver instead: you could install `broadcom-sta-dkms` on this machine as well. It should work similarly, but many T2 Linux users have reported better success with the `brcmfmac` + Apple firmware method ²⁴. If using STA, remember to disable Secure Boot and load `wl` as done on the iMac.)

Bluetooth: The `get-wifi-firmware` script also extracts Bluetooth firmware. After running it, reboot or reload the Bluetooth module: `sudo modprobe btusb`. The T2's Bluetooth (which might be part of the BCM4364 or a separate Apple chip) should then be recognized. Use `bluetoothctl` or Blueman to pair devices as usual.

5. Touch Bar Customization: By default, the Touch Bar will now function in “**Touch Bar Keyboard**” mode ²⁵ – meaning it basically shows the static row of function keys or media controls (like how Boot Camp does in Windows). If this is sufficient, you're all set. If you want to customize the Touch Bar with dynamic buttons (e.g. an escape key, or custom shortcuts), you can install `tiny-dfr`, a daemon that allows custom Touch Bar layouts ²⁶. Install it from the T2 repo:

```
sudo apt install tiny-dfr
```

Reboot, then run `sudo touchbar` – this interactive tool lets you configure the Touch Bar (e.g., choose function keys vs media keys, and other options) ²⁷. *Note:* full custom graphics on the Touch Bar are still being developed upstream (drivers for the Touch Bar display were in progress for newer kernels ²⁸). But `tiny-dfr` covers basic customization of the OLED buttons.

6. Finalize and Kernel Params: Lastly, add some recommended kernel parameters for T2 Macs. Edit `/etc/default/grub` and in `GRUB_CMDLINE_LINUX` add:

```
intel_iommu=on iommu=pt pcie_ports=native
```

Then run `sudo update-grub`. These parameters improve device enumeration on T2: enabling IOMMU and setting PCIe ports to native mode can help Thunderbolt and potentially NVMe stability ²⁹ ³⁰. (If Thunderbolt devices aren't working, some users have to switch this to `pcie_ports=compat` – the optimal setting can vary ³¹.)

At this point, with the T2 kernel and drivers, the MacBook Pro's internal devices should be operational: keyboard, trackpad, Touch Bar, Wi-Fi, BT, camera, audio, etc. Proceed to configuration and troubleshooting:

Option B: Installing T2 Drivers on Stock Kernel (Alternative)

If you chose not to use the `linux-t2` kernel, you will need to individually install drivers: The T2 repository provides DKMS modules: `apple-bce-dkms`, `apple-touchbar-dkms`, `applesmc-t2-dkms`, `bcm5974-t2-dkms` (for the trackpad), etc ³² ³³. You would install these and any dependencies, then add `apple-bce` to `/etc/modules` to load at boot ³⁴ (this driver is what exposes the keyboard, trackpad, etc. as USB devices and handles audio and camera bridging ²²). You'd also need the Wi-Fi firmware as above. However, since we used Option A, we won't detail every DKMS step here. The end result should be the same. (Using the pre-built kernel is simpler and recommended, as it avoids potential version mismatches and already includes the correct versions of all these drivers.)

Step 4: Configuration on MacBook Pro (Keyboard, Trackpad, Audio, etc.)

Now that hardware is enabled, configure it for optimal use:

- **Keyboard and Touchpad:** The internal keyboard and trackpad on T2 Macs appear as USB devices via `apple-bce`. The **trackpad** uses the `bcm5974` driver (same as older MacBooks, but the T2 repo

provides `bcm5974-t2` with improvements ³³). Multitouch and gestures (two-finger scroll, etc.) should work via the libinput driver. However, features like “force touch” and advanced haptics are not supported on Linux, and palm rejection might not be as good as macOS ³⁵ . You can tweak pointer speed and tap-to-click in MATE’s Mouse settings. If you find the tracking or touch sensitivity off, consider following the T2Linux **Trackpad tuning** guide ³⁶ which suggests libinput configurations to better mimic macOS feel. The **keyboard** works via the standard evdev/hid drivers. If you want the function (Fn) key behavior changed (F1-F12 vs media keys), use the `hid_apple fnmode` setting as described in the iMac section (it applies to this internal keyboard as well). The **keyboard backlight** on the MacBook Pro is controlled by the driver now; you can adjust it by pressing the Touch Bar brightness keys (in the default media strip) or via software: e.g., `echo <value 0-255> | sudo tee /sys/class/leds/smc::kbd_backlight/brightness`. MATE’s power settings can also auto-dim the keyboard backlight on idle (via UPower interfacing with applesmc).

- **Audio (Speakers and Microphone):** The T2 chip presents the audio device through the `apple-bce` driver as an internal USB audio interface. After installing `apple-t2-audio-config` and using the T2 kernel, ALSA/PulseAudio (or PipeWire) should detect an “Apple T2 Audio” output and input. On the 2019 MacBook Pro 16”, there are multiple speakers (including subwoofers) – the audio config package includes tuning for those (especially the 16-inch model has a special filterchain ³⁷). It’s recommended to use **PipeWire** on T2 Macs for better audio handling ³⁸ (Debian 12 still defaults to PulseAudio, which works, but PipeWire can be installed and set as default if desired). You may need to adjust the sound output balance due to the multi-speaker setup; the provided config should handle it, but test your speakers. The **internal microphone** should show up as well – however, note that some T2 users report low input volume. You can increase the gain in ALSA or PulseAudio settings. If you find audio crackling or not working after suspend, that’s a known issue – see Troubleshooting.
- **Camera (FaceTime HD):** On T2 Macs, the internal webcam is connected via the T2 and exposed by `apple-bce` as a USB camera device ²² . This is fortunate because it means the FaceTime camera might work as a standard UVC device now. Check `lsusb` for an Apple camera device or see if `/dev/video0` exists. In many cases, after installing `apple-bce`, the camera works out-of-the-box with the generic **uvcvideo** driver. Test it with Cheese or similar. If it doesn’t work, you may need the same **facetimehd** driver process as the iMac (it depends on how Apple implemented the T2’s camera interface). According to T2Linux, the camera should be exposed as USB and thus *should* work with UVC driver ³⁹ . If not, use the facetimehd approach described earlier to be sure.
- **Thunderbolt 3/USB-C:** Thunderbolt ports should work for USB and display outputs without special effort. If you have Thunderbolt 3 storage or eGPUs, Linux can use them, but hot-plug might be finicky. The kernel param we set (`pcie_ports=native` or `compat`) influences Thunderbolt enumeration ³¹ . If you find Thunderbolt devices aren’t recognized, try toggling that param (edit GRUB to use `pcie_ports=compat` if `native` was used, or vice versa, then update-grub and reboot) ⁴⁰ . This can fix Thunderbolt detection.
- **Power Management (Sleep, Battery, Fans):** This is a laptop, so power features are crucial:
- **Battery Monitoring:** The battery and AC adapter are managed by the SMC. With the T2 kernel, the `applesmc-t2` driver should be active (or compiled in) to expose battery status. You can check battery info via `upower -d` or in MATE’s battery indicator. It should show charge, rate, etc.
- **Sleep/Suspend:** Suspending a T2 MacBook is partly working but can be problematic. As of now, a recent macOS firmware update (macOS 14 “Sonoma”) has caused suspend issues on some T2 Macs ⁴¹ . Even before that, many users had to apply workarounds to get reliable suspend. The main issue is with the `apple-bce` driver during suspend/resume ⁴¹ . **Workaround:** A common fix is to unload the `apple-bce` module (and related HID modules for Touch Bar) right before suspend and reload

after resume ⁴². You can automate this with a systemd sleep script. For example, create `/etc/systemd/system-sleep/t2resume.sh` with:

```
#!/bin/bash
case "$1/$2" in
  pre/*)
    # Pre-suspend actions
    modprobe -r apple_bce ;;
  post/*)
    # Post-resume actions
    modprobe apple_bce ;;
esac
```

(For Touch Bar models, you might also remove `hid_applelb` modules and reinsert them in proper order as per T2Linux suggestions ⁴³, but often removing just `apple_bce` is enough for basic resume ⁴⁴.) Make the script executable. This will try to unload the driver on suspend (so the T2 doesn't hold onto devices) and reload on wake. Additionally, as with other Macs, you may need to disable USB wake triggers (`XHC1`) to prevent instant wake-ups (the MacBook's lid sensor should allow it to wake when opened, but USB devices might wake it unexpectedly). Use the `/proc/acpi/wakeup` technique:

```
sudo sed -i.bak -e '/XHC1/s/enabled/disabled/' /proc/acpi/wakeup
```

(This one-liner is not persistent; for a permanent solution, you could incorporate `echo XHC1 > /proc/acpi/wakeup` into the above sleep script or another init script.) After these tweaks, test suspend: close the lid or use `systemctl suspend`. The machine should sleep and hopefully resume when you open the lid or press a key.

- **Thermal and Fans:** The T2 chip also controls the fans. The `applesmc-t2` driver should expose fan speed control. You can use `macfanctld` or `t2fanrd` (a T2-specific fan daemon available in the repo: `sudo apt install t2fanrd`) to manage fan curves ⁴⁵. By default, the MacBook's fans might run a bit louder or stay low longer than on macOS. If you notice overheating, install a fan control daemon. T2FanRd in particular is made for T2 Macs and allows setting target temperatures.
- **Battery Life:** Linux might not achieve the same idle power draw as macOS on this hardware. You can improve it by using tools like **TLP** (`sudo apt install tlp`) for power tuning. This can auto-tune PCIe power states, USB autosuspend, etc. Also consider enabling SATA ALPM, PCIe ASPM, and use `power top --auto-tune`. Note that the AMD dGPU (if you have the 15"/16" model with Radeon) may consume power even when not in use, because dynamic GPU switching is not trivial. The `Hybrid Graphics` guide from T2Linux ⁴⁶ mentions that powering off the discrete GPU isn't working via software yet. It will always be on, using some power. A workaround is to force the dGPU into a lower power state: you can add `amdgpu.dpm=0` to kernel params or use `echo low > /sys/class/drm/card1/device/power_dpm_force_performance_level` to limit power ⁴⁷. This can prevent crashes and reduce heat if the AMD GPU is not actively used. On the bright side, the Intel iGPU can drive the internal display most of the time; using the AMD for rendering (via DRI_PRIME) is possible but may be unstable, so many simply let the AMD idle.

- **Secure Boot:** If you re-enable Secure Boot, note that the custom kernel and any DKMS modules are not signed by Microsoft's keys. You'd need to enroll your own Machine-Owner Key (MOK) to allow them, or just keep Secure Boot off (simplest solution).

Troubleshooting on the MacBook Pro (T2)

- **Keyboard/Trackpad not working after reboot:** If you rebooted into the new kernel and still can't use the internal keyboard/trackpad, double-check that the **T2 kernel is actually running** (uname should show a `t2` in the version). If not, select it from GRUB. If it is running but no input, it could be the `apple-bce` module didn't load. Try `sudo modprobe apple_bce`. If that fails, the module might not be present – ensure you installed the kernel correctly. In case of using the DKMS approach, verify `apple-bce` DKMS built and is inserted (`dkms status` and `lsmod`).
- **No Wi-Fi after firmware install:** If you used `get-wifi-firmware` and still see no Wi-Fi, run `dmesg | grep brcm`. If it says firmware file not found, the script may not have placed it correctly. Check `/lib/firmware/brcm/` for files like `brcmfmac4364*.bin`. If they're missing, rerun the script or try an alternate method (for instance, Boot Camp drivers: obtain the BootCampDrivers package for your Mac model from Apple, extract the Broadcom `*.bin` firmware and copy to `/lib/firmware/brcm/`). Another common pitfall is Secure Boot – if you opted for Broadcom STA driver and Secure Boot is enabled, the module `wl` will be blocked. Disable Secure Boot or sign the module with MOK. Also, ensure you removed or blacklisted the conflicting `b43/brcm80211` modules if using `wl`.
- **Audio issues (T2):** If sound output is distorted or some speakers not working, it could be an ALSA UCM configuration issue. The `apple-t2-audio-config` should handle mapping the channels correctly. Make sure you have PulseAudio or PipeWire using the correct profile (it should see an "Analog Stereo Output" for speakers). Use `alsamixer` to check if there are multiple output channels (e.g., sometimes the left/right might appear as separate outputs). If microphone input is too soft, install `pavucontrol` and boost the input level or mic boost if available. If audio fails after a suspend/resume, that's the bug mentioned; unloading `apple-bce` on suspend often helps, but if not, you may need to unload the ALSA USB audio module too. As a brute-force workaround, you can script a restart of PulseAudio on resume.
- **Webcam not detected:** If `lsusb` shows an Apple webcam device but no `/dev/video0`, ensure `uvcvideo` module is loaded (`sudo modprobe uvcvideo`). If the device still isn't created, you might have to use the `facetimehd` driver on T2 as well. Follow the same process as for the iMac. Some T2 models might still require it – check the T2Linux wiki or forums specific to your model if unsure.
- **Touch Bar not responding:** If the Touch Bar is dark or unresponsive, it may be the `apple-touchbar` driver didn't load. This driver should be built into the T2 kernel or installed as `apple-touchbar`. Try `sudo modprobe hid_appletb` (there are two modules: `hid_appletb_kbd` and `hid_appletb_bl` for keyboard and backlight respectively). If you can't get it to light up, check `dmesg` for errors. Remember that without additional software, it will only show the static keys – use `tiny-dfr` to customize.
- **Suspend or Shutdown hangups:** Some users report that the MacBook might hang on shutdown or reboot, possibly due to T2 drivers not unloading cleanly. If shutdown hangs, you can try adding a service to unload `apple_bce` on shutdown as well. In `/etc/modules-load.d/t2.conf` we already have `apple-bce` loading on boot³⁴; ensure you *don't* have duplicate drivers loaded. Also, make sure to update to the latest T2 kernel release – the project is active and may have fixed various issues in newer builds.

- **General System Stability:** Keep the system updated. T2 support is evolving, so newer kernels (or those from backports) might gradually include more upstream support. For instance, the Touch Bar drivers are slated to be upstream (in Linux 6.x) ²⁸, which means down the line Debian's official kernel might support more of this out-of-the-box. Until then, the community kernel is the way to go. If you encounter kernel panics or random reboots, note what triggers it – e.g., heavy disk I/O or using the AMD GPU – and consider updating the kernel or disabling the suspected component (for example, use the iGPU only, or if the crash happens on Wi-Fi use, try the wl driver instead). The cited T2Linux GitHub and wiki are great resources for troubleshooting specific quirks.

Direct Resources and Downloads

Below is a list of direct links and references to the tools and packages mentioned (for convenience, ensure you trust and verify sources before executing downloaded code):

- **Debian 12 Non-Free Installer ISOs:** [Debian Download – firmware included images](#).
- **Broadcom STA Driver (Debian package):** [broadcom-sta-dkms in Debian Packages](#) – install via apt as shown.
- **T2Linux Apt Repo and Kernel:** Instructions to add repository and packages ¹⁸ ²⁰. The repository is hosted on GitHub: [AdityaGarg8/t2-ubuntu-repo#](#) for T2 Macs. After adding, get the `linux-t2` and related packages via apt.
- **T2 Drivers Source (if needed):** [apple-bce](#), [apple-ib-drv \(Touch Bar interface\)](#) – these are the upstream projects, but using the apt/DKMS packages is easier.
- **Wi-Fi Firmware Extraction Script:** The `get-wifi-firmware` script from T2Linux (installed via `apple-firmware-script` package) – see T2Linux Wi-Fi guide ⁴⁸. Direct link to script on T2Linux wiki for reference (do not run blindly; use the packaged version): <https://wiki.t2linux.org/tools/firmware.sh>.
- **Facetime HD Camera Driver:** [patjak's bcwc_pcie GitHub](#) (driver source) and [facetimehd-firmware](#) (firmware extractor). There is also a pre-built DKMS in some repos (e.g., Arch AUR, and an outdated one in a PPA), but manual build ensures you get the latest.
- **Cirrus Logic CS8409 Audio driver for iMac/MacBook:** [davidjo/snd_hda_macbookpro](#) – contains patches and driver for Cirrus audio. Not needed for T2 (which uses USB audio), only for older machines like the iMac 18,3.

Conclusion

By following this guide, you should have Debian Trixie running with full functionality on both the 2017 iMac 5K and the 2019 MacBook Pro. We covered initial setup, required drivers, kernel modules, and troubleshooting of common issues. In summary:

- **iMac 5K (2017):** Use Debian's non-free support for Broadcom Wi-Fi (wl driver or firmware for `brcmfmac`), AMDGPU drivers for the 5K display, and install the `facetimehd` and Cirrus audio drivers for webcam and sound. Most features work with a bit of tweaking, as confirmed by community solutions ⁴ ⁹.
- **MacBook Pro (2019):** Leverage the community T2 patches – either via the custom kernel or DKMS modules – to enable the T2-managed devices (keyboard, trackpad, Touch Bar, audio, etc.) ⁴⁹ ⁵⁰. Retrieve Broadcom firmware for Wi-Fi/BT ⁵, and apply power-management workarounds for a smooth experience.

With the above steps, both systems can achieve feature parity with their macOS functionality: high-resolution graphics with acceleration, working Wi-Fi/Bluetooth, functional Touch Bar, working internal audio and camera, input devices with gestures and backlight, and proper power management (sleep, battery readings, etc.). Enjoy your fully enabled Debian MATE desktop on your Mac hardware!

Sources and References:

- Debian Wiki & Community Docs on installing Debian on Mac hardware and Broadcom drivers ⁵¹ ⁵² ⁵³ .
- T2Linux Project Wiki – *Installation and Setup Guides for Debian on T2 Macs* ⁴⁹ ²⁰ ²⁹ , *Wi-Fi/Bluetooth Firmware Guide* ⁴⁸ , *State of Hardware Support* ⁴⁰ ⁴¹ .
- Arch/Linux community forums – user experiences enabling Cirrus Logic audio on 2017 iMac ⁹ , Broadcom Wi-Fi on Macs ⁴ , and FaceTime HD camera support ¹¹ .
- T2Linux apt repository documentation – listing of provided packages (apple-bce, apple-touchbar, etc.) and their functions ⁵⁰ ⁵ .
- Josh Sherman's blog on MacBook suspend issues – solution for USB wake triggers ¹⁴ .

¹ ² ³ ²⁰ ²¹ ²⁹ ³⁸ ⁴⁹ Installation - t2linux wiki

<https://wiki.t2linux.org/distributions/debian/installation/>

⁴ Today's fresh install won't recognize iMac's Broadcom PCI wifi (or audio, either) - Network - Manjaro Linux Forum

<https://forum.manjaro.org/t/todays-fresh-install-wont-recognize-imacs-broadcom-pci-wifi-or-audio-either/94228>

⁵ ¹⁸ ¹⁹ ²² ³² ³³ ³⁷ ⁴⁵ ⁵⁰ GitHub - AdityaGarg8/t2-ubuntu-repo: APT repository for T2 Macs

<https://github.com/AdityaGarg8/t2-ubuntu-repo>

⁶ ⁷ ²³ ⁴⁸ Wi-Fi and Bluetooth - t2linux wiki

<https://wiki.t2linux.org/guides/wifi-bluetooth/>

⁸ ¹⁵ ²⁵ ²⁸ ³⁰ ³¹ ³⁵ ³⁶ ³⁹ ⁴⁰ ⁴¹ ⁴⁶ ⁴⁷ Device Support and State of Features - t2linux wiki

<https://wiki.t2linux.org/state/>

⁹ ¹⁰ [SOLVED] Sound does not work with 'dummy output' on iMac 18,2 / Newbie Corner / Arch Linux Forums

<https://bbs.archlinux.org/viewtopic.php?id=277990>

¹¹ From Faceless to FaceTime: installing webcam drivers on a Debian-powered MacBook Air | Andrea Fortuna

<https://andreafortuna.org/2024/08/24/from-faceless-to-facetime-installing-webcam-drivers-on-a-debian-powered-macbook-air>

¹² MacBook Air webcam (FacetimeHD) not working after upgrade to ...

<https://askubuntu.com/questions/1525187/macbook-air-webcam-facetimehd-not-working-after-upgrade-to-ubuntu-24-04-lts>

¹³ How to get Keyboard Light keys working on Macbook? - Ask Ubuntu

<https://askubuntu.com/questions/26248/how-to-get-keyboard-light-keys-working-on-macbook>

¹⁴ Getting suspend in Linux working on a MacBook Pro

<https://joshtronic.com/2017/03/13/getting-suspend-in-linux-working-on-a-macbook-pro/>

16 17 26 27 34 42 43 44 **Basic setup - t2linux wiki**

<https://wiki.t2linux.org/guides/postinstall/>

24 **Broadcom BCM4364 802.11ac Wireless Network Adapter**

<https://linux-hardware.org/?id=pci:14e4-4464-106b-07bf>

51 52 53 **Macbook Pro 2019 with BCM4364 802.11ac Wireless Network Adapter, with Debian 12 (Weekly):
But, NO WiFi - Super User**

<https://superuser.com/questions/1838613/macbook-pro-2019-with-bcm4364-802-11ac-wireless-network-adapter-with-debian-12>